

SoTA-методы построения многоуровневой памяти ИИ-агента

Срез: июль 2026

Обзор state-of-the-art методов: от оперативной памяти
до канонической архитектурной истины.
Сравнение 7 production-систем, 10 научных работ,
4 операции межуровневого взаимодействия.

1. Почему память — центральная проблема

Без многоуровневой памяти агент работает как человек с амнезией: каждая сессия — чистый лист. Он не помнит ваши предпочтения, повторяет одни и те же ошибки, не может накопить экспертизу в предметной области.

Цифры (2026):

- 65% провалов enterprise-агентов — потеря контекста, а не ограничения модели (Gartner)
- 57% организаций уже используют агентов в production, но 32% называют качество главным барьером

Три подхода к решению

Подход	Кто делает	Идея	Плата
Инфраструктура	Mem0, Zep, Cognition	Система сама извлекает факты из диалогов	Дорого на запись, чисто на чтение
Агент управляет	Letta (MemGPT)	Агент решает что сохранить через итерации	Зависит от дисциплины агента
Сжатие + поиск	SimpleMem, mcp-memory	История сжимается, не хранится полностью	Потери при сжатии

2. Пять уровней памяти (от быстрой к глубокой)

Представьте память агента как слоёный пирог. Чем глубже слой — тем медленнее доступ, но тем надёжнее и долговечнее знания.

Уровень 0: Оперативная память (Working Memory)

Контекстное окно — то, что агент «видит» прямо сейчас. Текущий диалог, последние результаты.

- **Скорость:** мгновенно (прямой доступ модели)
- **Объём:** ограничен бюджетом токенов (1–200 тысяч)
- **Время жизни:** одна сессия, стирается при закрытии
- **Формат:** свободный текст + JSON-структуры
- **Надёжность:** низкая (непроверенное, временное)

Кто так делает: все агенты. Но проблема — когда это единственная память, агент буквально «просыпается без воспоминаний».

Уровень 1: Событийная память (Episodic)

Журнал событий. Что происходило: запросы пользователя, вызовы инструментов, результаты, обратная связь.

- **Скорость:** миллисекунды (поиск по индексу)
- **Объём:** неограничен (с ротацией)
- **Время жизни:** сессии недели месяцы
- **Формат:** размеченные по времени структурированные записи
- **Надёжность:** средняя (автоматическая запись без проверки интерпретации)

Кто так делает: Zep/Graphiti — единственная система, различающая «когда событие произошло» и «когда система о нём узнала». Letta Recall Memory — поиск по истории разговоров. LangMem Episodic — успешные взаимодействия как примеры для будущего.

Уровень 2: Фактографическая память (Semantic)

«Энциклопедия» агента. Обобщённые факты и знания — не конкретные события, а то, что из них извлечено.

- **Скорость:** десятки миллисекунд (поиск по смыслу + граф)
- **Объём:** неограничен
- **Время жизни:** пока актуально
- **Формат:** типизированные сущности, структурированные факты
- **Надёжность:** высокая (выведено из многократных наблюдений или проверено вручную)

Переход от событийной к фактографической памяти — ключевой процесс: из десятков сообщений «пользователь использует Python» формируется один факт «основной язык: Python».

Кто так делает:

- Mem0 (48K звёзд) — гибрид трёх хранилищ: векторное, ключ-значение, графовое. Автоматически извлекает факты через LLM.
- Letta Archival Memory — неограниченное внешнее хранилище
- LangMem Semantic — коллекции для поиска + профили с обновлением на месте
- A-MEM (NeurIPS 2025) — метод Zettelkasten: атомарные карточки знаний со связями

Уровень 3: Процедурная память (Procedural)

«Рецепты» — как делать. Рабочие процессы, методы, правила, скиллы.

- **Скорость:** предзагружается в контекст или миллисекунды
- **Объём:** компактный (правила, не данные)
- **Время жизни:** пока метод актуален
- **Формат:** исполнимые инструкции, структурированные workflows
- **Надёжность:** высокая (откалибровано многократным исполнением)

Ключевое различие: описание метода $\neq$ сам факт работы. Рецепт пирога $\neq$ испечённый пирог. Процедурная память — это рецепт.

Кто так делает: LangMem Procedural (паттерны эволюционируют на основе обратной связи), Claude Code AutoDream (консолидация паттернов в CLAUDE.md).

Уровень 4: Каноническая память (Canon)

«Конституция» — архитектурная истина. Принципы, фундаментальные различия, source-of-truth для платформ.

- **Скорость:** сотни миллисекунд (комбинированный поиск + перекрёстная проверка)
- **Объём:** ограничен рецензированием
- **Время жизни:** пока платформа жива
- **Формат:** формальные модели, спецификации
- **Надёжность:** максимальная (рецензировано, многократно перепроверено)

Кто так делает: Pack-архитектура (иерархия DS Pack Base), Zep Community Subgraph (высокоуровневые тематические кластеры), A-MEM Zettelkasten (структура связей как самостоятельная ценность).

3. Как уровни работают вместе: 4 операции

Операция 1: Сжатие при подъёме (Compress)

Когда данные переходят с уровня на уровень, они сжимаются. Важно — сжатие должно быть явным: что сохранено, что потеряно, для чего можно использовать результат.

Пример цепочки:

Полный диалог (Working) Краткое саммари + извлечённые факты (Episodic)
 Обобщённый факт «Предпочитает Python» (Semantic)
 Pack-сущность с этим знанием (Canon)

Кто так делает:

- Claude Code AutoDream: удаление устаревшего, слияние связанного, обновление структуры
- LangMem Subconscious: анализ после разговора, ноль влияния на скорость
- Mem0 auto-extraction: LLM-пайплайн извлекает факты атомарно
- D-MEM: двухпроцессная модель — быстрый путь (кэш) + медленный (эволюция на ошибках)
- MemoryOS (EMNLP 2025): иерархия shortmidlong, +48% точности

Операция 2: Выбор при загрузке (Select)

Когда агенту нужно решить задачу, он не загружает всю память — только релевантные части. 4 стратегии Context Engineering (Anthropic 2025, Gartner назвал «новой дисциплиной 2026»):

Стратегия	Что делает	Пример
-----------	------------	--------

Write	Создавать правильные файлы для контекста	CLAUDE.md, memory/
Select	Выбирать релевантное под задачу	Только нужные Раск-сущности
Compress	Сжимать до необходимого минимума	Саммари ≤100 строк вместо диалога
Isolate	Изолировать контексты друг от друга	Разные репо — разные CLAUDE.md

Ключевой инсайт (OpenAI, 2026): изменение только обвязки (не модели) улучшило результаты 15 разных LLM на 5–14 п.п. Обвязка важнее модели.

Три паттерна из практики OpenAI:

- Progressive Disclosure: агент начинает с маленькой стабильной точки входа, учится где искать дальше
- Co-located Plans: планы, задачи, техдолг — всё версионировано рядом с кодом
- Agent Legibility: код и документация навигабельны для агента без внешних подсказок

Операция 3: Устаревание и забывание (Decay)

Самая нерешённая проблема в индустрии. Только 2 системы (Zep и Hindsight) имеют явный механизм устаревания памяти. Остальные либо хранят всё бесконечно, либо затирают без разбора.

Что нужно:

- Окна достоверности: «этот факт был истинным с марта по июнь 2026»
- Снижение важности со временем: редко используемое менее важное
- Штраф за устаревание: старая непроверенная информация — понижаем доверие
- Дисциплинированное забывание: явная отставка с пометкой «устарело», не молчаливое удаление

Кто решил: Zep bi-temporal модель — Event Time (когда факт стал истинным) + Ingestion Time (когда система узнала). Можно запросить «что мы знали о X в момент T?»

Операция 4: Память для нескольких агентов (Multi-agent)

Когда агентов больше одного, возникает несколько проблем:

- Кто что видит: каждый агент должен видеть только свой домен
- Общая память: что разделяется между агентами
- Одновременная запись: как не испортить данные при параллельной работе — открытая проблема
- Проверка: как убедиться, что память не повреждена

Рабочий подход — модель Парламента: координатор + N доменных агентов + верификатор с доступом только на чтение. Staged rollout (4 фазы, ≥30 дней стабильности каждая): Дворецкий Рабочий стол Допуски Команды.

4. Сравнение систем (июль 2026)

Система	GitHub	Тип памяти	Экстракция	Поиск	Decay	Multi-agent
Letta	~21K	3 уровня	Агент сам	Агент-упр.	Агент-контр.	Да
Mem0	~48K	Факты + граф	Авто (LLM)	Vector+Graph	Нет	Да
Zep/Graphiti	~20K	Соб/Факты/Темы	Авто-entity	Время+смысл	Да!	Да
LangMem	—	3 типа (CoALA)	Авто (фон)	По смыслу	Нет	Да
Claude Code	—	Markdown	AutoDream	Прямое чтение	Pruning	Нет

Khoj	~10K	Индекс док.	Индексация	По смыслу	Нет	Нет
COG 2nd brain	—	Git-md	Вручную	По файлам	Git history	Нет

5. Ключевые научные работы

Работа	Год	Суть
CoALA (Princeton)	2023	Стандартная таксономия: рабочая/событийная/фактографическая/процедурная
MemGPT Letta (UCB)	2023-25	Метафора ОС: агент сам управляет памятью через инструменты
MemOS (SJTU)	2025	Единая модель: параметрическая + активационная + текстовая. +159% temporal reasoning
MemoryOS (EMNLP)	2025	Иерархия shortmidlong. +48% F1 на LoCoMo
A-MEM (NeurIPS)	2025	Метод Zettelkasten для ИИ: атомарные карточки знаний со связями
D-MEM	2026	Двухпроцессная: быстрый путь ($O(1)$) + медленный (обучение на ошибках)
Zep Temporal KG	2024	Единственная система с bi-temporal окнами достоверности
Survey 2512.13564	2025	Полный обзор — пробелы: decay, multimodal, multi-agent writes
MemMA	2026	Координация памяти для нескольких агентов
Harness Eng. (OpenAI)	2026	+5-14 п.п. без смены модели — только обвязка

6. Что остаётся нерешённым

- **Устаревание и свежесть:** Только Zep и Hindsight решили. Остальные игнорируют.
- **Память в весах модели:** Перспективно в академии, отсутствует в продуктах.
- **Параллельная запись агентов:** Нерешённая задача.
- **Стоимость авто-извлечения:** Всё ещё требует LLM-вызовов для каждой операции.
- **Доверие к памяти:** Как проверить, что в памяти соответствует реальности?
- **Память правильное действие:** Наличие памяти не гарантирует правильное поведение.
- **Дисциплина забывания:** Когда и что удалять — не формализовано.

7. Как спроектировать многоуровневую память: пошаговый метод

Шаг 1. Определите уровни

Для каждого уровня ответьте на 6 вопросов:

- О чём этот уровень? (сущности какого типа хранятся)
- В каком окружении он работает? (рантайм, Git, база данных)
- Что именно хранится? (утверждения какого рода)
- Как это читать? (формат, схема)
- С какой точки зрения? (роль агента, пользователя, системы)
- Какое представление порождается? (файлы, записи, индексы)

Шаг 2. Присвойте характеристики каждому уровню

- Структурированность: от свободного текста до машинопроверяемого

- Область применимости: сессия проект домен платформа
- Надёжность: от «только что записано» до «рецензировано и многократно проверено»

Шаг 3. Объявите переходы между уровнями

Для каждой пары уровней: как данные переходят с нижнего на верхний? Что теряется при сжатии? Что сохраняется?

Шаг 4. Настройте устаревание

Для каждого типа записей: какой срок жизни? Когда перепроверить? Что делать, если информация устарела?

Шаг 5. Примените стратегии Context Engineering

- Write: создайте правильные файлы для каждого уровня
- Select: определите, что загружается в контекст для разных типов задач
- Compress: для каждого перехода — что сжимается и с какими потерями
- Isolate: разделите память по доменам

Пример: фактографическая память для персональной AI-системы

Уровень: Фактографический

Хранит: 50 Раск-сущностей + 200 заметок + 30 характеристик доменов

Формат: типизированные сущности, структурированный JSON

Область: все домены платформы

Надёжность: 0.75 (из многих событий + review)

Переходы:

Событийная Фактографическая: теряется порядок событий,
метаданные сессии; сохраняется факт, сущность, отношение

Фактографическая Каноническая: знания из заметок

Раск-сущности с авторским review

Хранение: PostgreSQL (JSONB) + pgvector + граф сущностей

Устаревание: 30-дневное окно достоверности, удаление по неисп-нию

8. Основные выводы

- **1. Одной оперативной памяти недостаточно.** Без событийной и фактографической — агент не учится.
- **2. Главный нерешённый вопрос — устаревание.** Большинство систем либо хранят всё бесконечно, либо удаляют без разбора. Zer — единственный с явным решением.
- **3. Контекст важнее модели.** OpenAI показал: изменение того, что агент «видит», даёт +5–14 п.п. без замены модели.
- **4. Сжатие должно быть явным.** Вы должны знать, что потеряно при переходе с уровня на уровень, а не гадать.
- **5. Многоагентная память — открытый фронт.** Одновременная запись нескольких агентов — нерешённая задача.

Источники: CoALA (2309.02427), MemGPT (2310.08560), MemOS (2505.22101 / 2507.03724), MemoryOS (2506.06326), A-MEM (2502.12110), D-MEM (2603.14597), Zep (2501.13956), Survey (2512.13564), MemMA (2603.18718). Anthropic Context Engineering (2025), OpenAI Harness Engineering (2026), State of Agent Memory (Mohindra, 2026). Системы: Letta, Mem0, Graphiti, LangMem, Khoj, COG second brain.

Документ собран в июле 2026. Написан без специальной терминологии — для понимания с нуля.